Lesson Notes

MODULE 3 | LESSON 2

DEEPVAR: THE PYTHON IMPLEMENTATION

Reading Time	35 minutes
Prior Knowledge	Long Short Term Memory (LSTM), Recurrent Neural Network (RNN), Maximum Likelihood Estimation (MLE), Value at Risk (VaR)
Keywords	GluonTS library, DeepAR model

In the previous lesson, we learned about this powerful RNN model for probabilistic forecasting that can be used for portfolio VaR estimation; we learned the theory and the relevant algorithms. In this lesson, we walk through the GitHub repo using the example of multiple FX time series.

1. Probabilistic Forecasting

In the first code block, we import several modules from the `gluonts` library. GlounTS is a toolkit for time series modeling and includes some pre-built models, mostly focused on probabilistic forecasting. As GluonTS indicates, probabilistic forecasts "are predictions in the form of a probability distribution, rather than simply a single point estimate" (qst). If our goal was simply to predict tomorrow's price, that would be just a single point estimate and we would not need the whole distribution. However, since our goal is a Value at Risk estimate for a given set of quantiles (for example, .95, .99, and, as we will see, .05 and .01), we in fact require the entire distribution.

Code blocks two through eight deal with getting, cleaning, formatting, transforming, and displaying the data that will be used to train our model.

2. DeepAR (a Central Component to DeepVaR)

2.1 DeepAR Estimator

In code block nine, we create the class DeepARModel. This class has a method `fit`, which constructs the `DeepAREstimator` instance, which takes many parameters (as listed in the documentation for the package gluonts.model.deepar), the most important for our purposes being the following:

- **freq** – Frequency of the data to train on and predict
- **prediction_length** – Length of the prediction horizon
- **trainer** – Trainer object to be used (default: Trainer())
- **context_length** – Number of steps to unroll the RNN for before computing predictions (default: None, in which case context_length = prediction_length)
- **num_layers** – Number of RNN layers (default: 2)
- **num_cells** – Number of RNN cells for each layer (default: 40)
- **cell_type** – Type of recurrent cells to use (available: 'lstm' or 'gru'; default: 'lstm')
- **dropout_rate** – Dropout regularization parameter (default: 0.1)
- **distr_output** – Distribution to use to evaluate observations and sample predictions (default: StudentTOutput())

(Excerpted from "GluonTS.Model.Deepar Package.")

All of the above are explicitly parameterized in the code (except for the last one) and you can see the values there; the purpose of listing these here is primarily to show the meaning of each, as well as to point out the default indicated for the last parameter.

2.2 DeepAR Training: Likelihood and Optimization

Neither the paper nor the code indicate any distributional assumptions, only that it "consists of a product of likelihood factors" which are "maximized to train a deep neural network (RNN) that learns the distributions' parameters" (Fatouros 7). In order to learn the distributions' parameters, however, we need to assume a likelihood distribution. We can see above that the model defaults to the Student's t distribution for this likelihood calculation.

In fact, the training for the model consists entirely of finding the parameters for this distribution which make the actual training data the most likely. In practice, this is accomplished by maximizing the log likelihood. For the training of the DeepAR model, this equation provided by Salinas et al. (5) in their paper introducing DeepAR captures the task at hand:

$$\mathscr{L} = \sum_{i=1}^{N} \sum_{t=t_0}^{T} \log \left(z_{i,t} | \theta \left(\mathbf{h}_{i,t} \right) \right)$$

The cursive l is the likelihood for the time series represented by z whose subscripts i and t refer to the instrument and time step, respectively. The /theta includes all the learned parameters for the deterministic function **h** with the same subscripted indices as z (Salinas et al. 5). These authors note that h(it) is indeed deterministic since it is a function of entirely observed variables. To restate this, there are no latent variables involved in this calculation, so there is no inference required. The upshot of these observations is that the function **h** "can be optimized directly via stochastic gradient descent by computing gradients with respect to" the parameters of the model (Salinas et al. 5). It is not obvious from the code in this repo, but it was mentioned in the paper (Fatouros et al. 10) that DeepVaR uses the Adam optimizer. Indeed, you can confirm in the source code for the `Trainer` imported in the first block (e.g., `from gluonts.mx.trainer import Trainer`) that indeed `optimizer = mx.optimizer.Adam`.

Once the "Estimator" object has been fit, this object represents not just the type of forecasting model and relevant arguments, but also all the coefficients, hyperparameters, and weights, which fully specify the model to be used for probabilistic forecasting.

3. DeepVaR's Portfolio VaR

Recall the Portfolio VaR function from the repo:

```
def var_p(predictions, returns, weights, days_ahead=0, alpha=95):

    V = np.zeros(len(weights))

    for i in range(len(weights)):

      if weights[i] < 0 :

          V[i] = weights[i] * np.percentile(predictions[i].samples[:,days_ahead], a

      else:

          V[i] = weights[i] * np.percentile(predictions[i].samples[:,days_ahead], 1

    R = returns.corr()

    return -np.sqrt(V @ R @ V.T)
```

(Fatouros et al.)

The DeepVaR's portfolio VaR function has at least two elements worthy of note:

1. The treatment of long and short positions is very straightforward, in that when a weight is positive, the numpy.percentile function takes 100-/alpha as an input. When /alpha is 99 (corresponding to a 99% VaR), the function will return the prediction associated with the the very low end of samples, as we would expect. A high-alpha VaR, which is the kind we are most likely interested in, should correspond to low or negative returns. Conversely, a short position in an instrument, evidenced by a negative weight, means that we are most interested in (or worried about) a high return in that instrument because that equates

to a loss on our short position. The code handles this straightforwardly, so long as one is clear about the meaning of alpha and its (inverse) relationships to return percentiles and VaR confidence.

2. The treatment of correlation in the repo, on the other hand, seems very simplistic and actually at odds with the guidance in the paper to use the correlation for the preceding 125-day window. Salinas et al. assert that their model "effectively learns a global model from related time series, handles widely-varying scales through rescaling and velocity-based sampling, generates calibrated probabilistic forecasts with high accuracy, and is able to learn complex patterns such as seasonality and uncertainty growth over time from the data" (9). However, a single correlation measure calculated once for the entire time series of all the returns cannot adequately capture the non-stationarity of correlation across instruments and its impact on instrument returns, not to mention portfolio returns. Using a single "average" correlation across the entire time series in the portfolio VaR calculation does not reflect the reality that correlations change drastically in markets with turmoil. Let's not forget that VaR's focus on the *losses* end of the PnL distribution means that these turmoil correlations need special attention. It is also worth noting at this point what appears to be a pair of typos in the research paper by Fatouros et al., as they mention using the covariance instead of the correlation in both the text and in Algorithm 1 (8 and 9, respectively), though elsewhere they specify the correlation matrix. The correlation matrix does indeed seem more appropriate because the variance of the instruments, which would be captured by the covariance matrix, is already reflected in *V*, the weighted VaR of each individual instrument.

$$VaR_p = \sqrt{VRV^T}$$

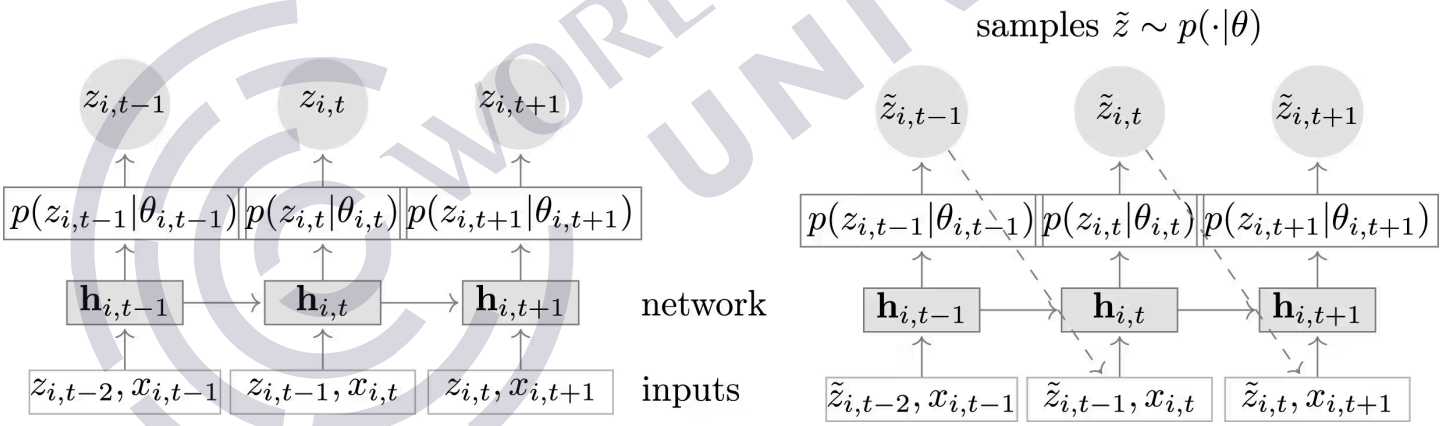
where V is a vector of the weighted VaR estimates per instrument,

$$V = [w_{AUDUSD}VaR_{AUDUSD}, w_{GBPUSD}VaR_{GBPUSD}, w_{USDJPY}VaR_{USDJPY}, w_{EURUSD}VaR_{EURUSD}]$$

and R is the correlation matrix.

In fact, although the DeepAR model learns much about correlations from its joint sampling of time series, why not provide the correlation structure explicitly as a covariate input? The original DeepAR model allows for covariates as you can see in this visualization from the DeepAR paper, which differs from the visualization in the DeepVaR paper and repo in one respect: the presence of covariates as inputs. The other minor difference is Salinas et al. use *p* to emphasize the modeling of the conditional distribution, where Fatouros uses the cursive l emphasizing the likelihood function (to model the same), which is discussed above.

Figure 2: Covariate Inputs to Conditional Distribution Modeling in DeepVaR's Recursive RNN



Source: Salinas, David, et al. "DeepAR: Probabilistic Forecasting with Autoregressive Recurrent Networks." *arXiv*, 13 Apr 2017, p. 3, https://www.sciencedirect.com/science/article/pii/S0169207019301888?via%3Dihub.

Can you review the DeepVaR code and make note of where changes would have to be made to 1) calculate and 2) incorporate the recent correlation across instruments (with the same context length as the rest of the sample) as a covariate input to the DeepVaR model as it is currently presented in GitHub? Can you write the actual code to make these modifications?

Separately, can you rewrite the code in the `var_p` function to use the 125-day rolling correlation for the calculation of the portfolio VaR, instead of a single correlation that spans the entire returns series?

Write an essay describing how you can test whether these changes improve the probabilistic forecasts and VaR estimates. How would you test these changes? Include code or pseudocode, and/or the results of any experiments your perform.

4. Conclusion

In these first two lessons, we looked at using neural net technology to better forecast Value at Risk. In the next two lessons, we learn about and apply state-of-the-art transformer technology with the same aim.

References

- Fatouros, Georgios, et al. "DeepVaR: A Framework for Portfolio Risk Assessment Leveraging Probabilistic Deep Neural Networks." *SpringerLink*, 13 April 2022, https://link.springer.com/article/10.1007/s42521-022-00050-0.
- Fatouros, Georgios. "DeepVaR: Portfolio Risk Assessment leveraging Probabilistic Deep Neural Networks." GitHub, https://github.com/giorgosfatouros/DeepVaR.
- "GluonTS.Model.Deepar Package." *GluonTS.ai*, https://ts.gluon.ai/stable/api/gluonts/gluonts.model.deepar.html.
- Salinas, David, et al. "DeepAR: Probabilistic Forecasting with Autoregressive Recurrent Networks." *arXiv*, 13 Apr 2017, https://www.sciencedirect.com/science/article/pii/S0169207019301888?via%3Dihub.